

UNIVERSIDAD POLITÉCNICA DE CHIAPAS

ALUMNO : CARLOS JAFFET VICENTE RINCON

INGENIERA DE SOFTWARE

MATERIA : PROGRAMACION PARA MOVILES II

ACTIVIDAD : INVESTIGACION -> GESTION
DE ESTADOS

24 DE MAYO DEL 2026 EN SUCHIAPA ,
CHIAPAS



INDICE

3. Introducción	3
4. Marco Teórico	3
Definición de Gestión de Estados en Flutter	3
Diferencias entre Estado Local y Global	3
5. Desarrollo del Tema	4
setState()	4
InheritedWidget / InheritedModel	5
Provider	6
Riverpod	7
BLoC / Cubit (Business Logic Component)	8
GetX	9
MobX	10
6. Análisis Comparativo	12
Cuadro Comparativo de Métodos de Gestión de Estado	12
Recomendación Técnica sobre qué Método Elegir	13
7. Conclusiones	14
8. Referencias	14

3. Introducción

En el desarrollo de aplicaciones móviles modernas, la interactividad y la reactividad son pilares fundamentales para ofrecer una experiencia de usuario óptima. Dentro de este ecosistema, Flutter se ha consolidado como un framework de desarrollo multiplataforma líder gracias a su arquitectura basada en widgets y su motor de renderizado de alto rendimiento. Sin embargo, a medida que una aplicación crece en complejidad, el flujo de datos y la sincronización entre los distintos componentes de la interfaz de usuario se vuelven considerablemente más difíciles de administrar. La gestión de estados constituye el núcleo arquitectónico de cualquier aplicación en Flutter. Determina cómo se procesan los eventos del usuario, cómo fluyen los datos a través de las diferentes capas del software y de qué manera la interfaz visual reacciona ante los cambios de la lógica de negocio. Una gestión de estados deficiente no solo provoca problemas de rendimiento debido a reconstrucciones innecesarias de la interfaz, sino que también degrada la mantenibilidad del código, dificultando la implementación de pruebas unitarias y la escalabilidad del sistema. Por lo tanto, comprender y seleccionar el enfoque adecuado para administrar el estado es una decisión crítica para cualquier ingeniero de software.

4. Marco Teórico

Definición de Gestión de Estados en Flutter

En Flutter, la interfaz de usuario es reflexiva y depende por completo del estado actual de la aplicación; esto significa que la interfaz visual se comporta como una función directa de los datos existentes en un momento determinado. El estado se define como cualquier información que puede cambiar a lo largo de la vida útil de la aplicación y que afecta de manera directa al comportamiento o a la representación visual de los componentes. La gestión de estados es, por consiguiente, el conjunto de patrones arquitectónicos, técnicas y herramientas que se encargan de almacenar, actualizar y distribuir esta información de manera eficiente a lo largo de toda la estructura de la aplicación.

Diferencias entre Estado Local y Global

- **Estado Local (Ephemeral State):** Es aquel estado que está contenido de manera exclusiva dentro de un único widget o que no necesita ser compartido

con otros componentes distantes del árbol. Su ciclo de vida es corto y está estrechamente ligado al widget que lo aloja. Un ejemplo clásico es la página actual de un PageView, una animación en progreso o el texto temporal introducido en un campo de entrada de datos.

- **Estado Global (App State):** Es la información que debe compartirse a través de múltiples pantallas, componentes o servicios de la aplicación. Su ciclo de vida suele persistir durante toda la sesión del usuario. Ejemplos de este tipo incluyen las credenciales de autenticación, las preferencias de configuración del sistema (como el modo oscuro), el contenido de un carrito de compras o los datos almacenados en caché provenientes de un servidor web.

5. Desarrollo del Tema

setState()

Es el mecanismo nativo y más elemental provisto por la clase State en Flutter para gestionar el estado local. Al invocarlo, se notifica al framework que los datos internos de un StatefulWidget han cambiado, lo que agenda de inmediato una reconstrucción de dicho widget y de sus descendientes en el siguiente ciclo de renderizado.

```
import 'package:flutter/material.dart';

class CounterScreen extends StatefulWidget {
  const CounterScreen({super.key});

  @override
  State<CounterScreen> createState() =>
    CounterScreenState();
}

class _CounterScreenState extends State<CounterScreen> {
  int _counter = 0;

  void _incrementCounter() {
    setState(() {
      _counter++;
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Center(
        child: Text(
          '$_counter',
          style: const TextStyle(fontSize: 24),
        ),
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: _incrementCounter,
        child: const Icon(Icons.add),
      ),
    );
  }
}
```

- **Ventajas:** No requiere librerías externas; curva de aprendizaje nula; ideal para prototipos rápidos y animaciones locales sencillas.
- **Desventajas:** Reconstruye todo el subárbol del widget, afectando el rendimiento en jerarquías complejas; mezcla la lógica de negocio con la interfaz de usuario; inviable para el manejo de estados globales.
- **Casos de uso:** Formularios simples, interruptores de UI (toggles), animaciones efímeras y componentes aislados sin dependencias externas.

InheritedWidget / InheritedModel

InheritedWidget es una clase base nativa que permite propagar información de manera descendente a lo largo del árbol de widgets. Los widgets hijos situados en niveles inferiores pueden acceder a las propiedades expuestas utilizando el contexto, registrándose automáticamente para recibir notificaciones cuando el estado del ancestro se actualiza.

```
import 'package:flutter/material.dart';

class AppThemeData {
  final Color primaryColor;
  AppThemeData({required this.primaryColor});
}

class CustomThemeProvider extends InheritedWidget {
  final AppThemeData themeData;

  const CustomThemeProvider({
    super.key,
    required this.themeData,
    required super.child,
  });

  static CustomThemeProvider? of(BuildContext context) {
    return context.dependOnInheritedWidgetOfExactType<CustomThemeProvider>
    ();
  }

  @override
  bool updateShouldNotify(CustomThemeProvider oldWidget) {
    return themeData.primaryColor != oldWidget.themeData.primaryColor;
  }
}
```

- **Ventajas:** Solución integrada en el core de Flutter; excelente rendimiento para la lectura de datos pasivos; evita el paso manual de propiedades a través de constructores (prop drilling).
- **Desventajas:** Sintaxis sumamente verbosa; no proporciona un mecanismo directo e intuitivo para mutar o actualizar el estado desde los widgets hijos.
- **Casos de uso:** Propagación de configuraciones estáticas globales, como temas visuales personalizados o configuraciones de localización e idiomas.

Provider

Construido como una capa de abstracción sobre InheritedWidget, Provider simplifica drásticamente la gestión de estados intermedios y globales. Utiliza patrones de diseño comunes en la programación orientada a objetos, apoyándose en ChangeNotifier para emitir alertas cuando ocurren cambios en los datos.

```

import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

class AuthProvider extends ChangeNotifier {
  String? _token;

  String? get token => _token;

  void login(String newToken) {
    _token = newToken;
    notifyListeners();
  }
}

class UserProfile extends StatelessWidget {
  const UserProfile({super.key});

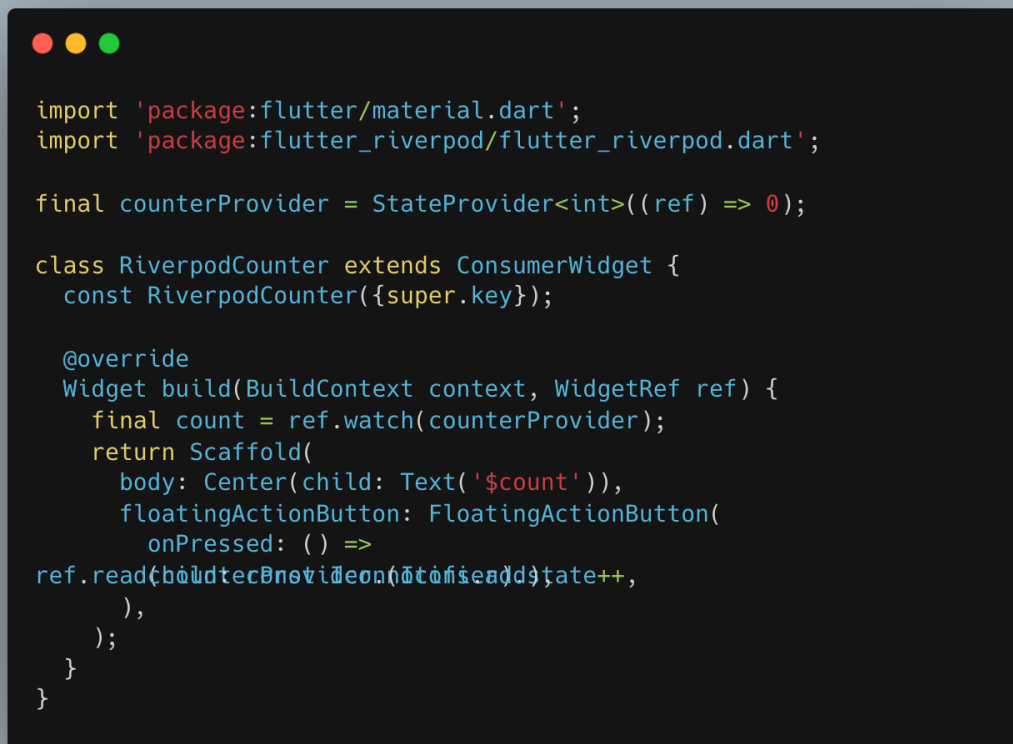
  @override
  Widget build(BuildContext context) {
    final auth = Provider.of<AuthProvider>(
      context,
    );
    return Text(auth.token ?? 'No autenticado');
  }
}

```

- **Ventajas:** Sintaxis limpia y fácil de adoptar; recomendada históricamente por el equipo de Flutter; integración nativa limpia con el ciclo de vida de los widgets.
- **Desventajas:** Depende estrictamente del BuildContext, dificultando su uso fuera de la capa de UI; errores en tiempo de ejecución si se intenta acceder a un proveedor no registrado en el árbol.
- **Casos de uso:** Aplicaciones de tamaño pequeño a mediano con arquitecturas estándar basadas en el patrón MVVM.

Riverpod

Diseñado por el mismo creador de Provider, Riverpod es una reescritura completa que elimina las limitaciones técnicas asociadas a InheritedWidget. No depende en absoluto del árbol de widgets ni del contexto para almacenar o acceder al estado, resolviendo las referencias en tiempo de compilación.



```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

final counterProvider = StateProvider<int>((ref) => 0);

class RiverpodCounter extends ConsumerWidget {
  const RiverpodCounter({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final count = ref.watch(counterProvider);
    return Scaffold(
      body: Center(child: Text('$count')),
      floatingActionButton: FloatingActionButton(
        onPressed: () =>
          ref.read(counterProvider.notifier).state++,
      ),
    );
  }
}
```

- **Ventajas:** Seguridad en tiempo de compilación (elimina los errores de tipo `ProviderNotFoundException`); completamente independiente del contexto de Flutter; facilita enormemente las pruebas unitarias.
- **Desventajas:** Curva de aprendizaje moderada; introduce conceptos y tipos de proveedores específicos que requieren familiarización.
- **Casos de uso:** Aplicaciones profesionales modernas que requieran alta modularidad, escalabilidad estricta y pruebas automatizadas robustas.

BLoC / Cubit (Business Logic Component)

BLoC es un patrón arquitectónico maduro fundamentado en la programación reactiva mediante flujos de datos (Streams). Separa de forma tajante la interfaz de usuario de la lógica de negocio. Los widgets envían eventos fijos hacia el BLoC, y

este procesa la información para emitir nuevos estados secuenciales. Cubit es una variante simplificada que reemplaza los eventos por funciones tradicionales.

```
import 'package:flutter_bloc/flutter_bloc.dart';

abstract class AuthState {}
class AuthInitial extends AuthState {}
class AuthSuccess extends AuthState { final String user; AuthSuccess(this.user); }
class AuthCubit extends Cubit<AuthState> {
  AuthCubit() : super(AuthInitial());

  void authenticateUser(String username) {
    emit(AuthSuccess(username));
  }
}
```

- **Ventajas:** Separación absoluta de responsabilidades; predecible y altamente estructurado gracias a la trazabilidad de sus estados; ideal para equipos grandes de desarrollo.
- **Desventajas:** Requiere una gran cantidad de código base (boilerplate) para mapear eventos y estados; puede percibirse como sobreingeniería en proyectos simples.
- **Casos de uso:** Aplicaciones empresariales de gran escala, sistemas bancarios o proyectos con flujos de datos complejos que exijan auditorías estrictas de estado.

GetX

GetX se presenta como una solución integral de alto rendimiento que combina la gestión de estados reactiva con un inyector de dependencias integrado y un gestor de rutas nativo, todo ello prescindiendo por completo del uso de BuildContext.

```

import 'package:get/get.dart';
import 'package:flutter/material.dart';

class TaskController extends GetxController {
  var taskCount = 0.obs;

  void increment() => taskCount++;
}

class TaskView extends StatelessWidget {
  const TaskView({super.key});

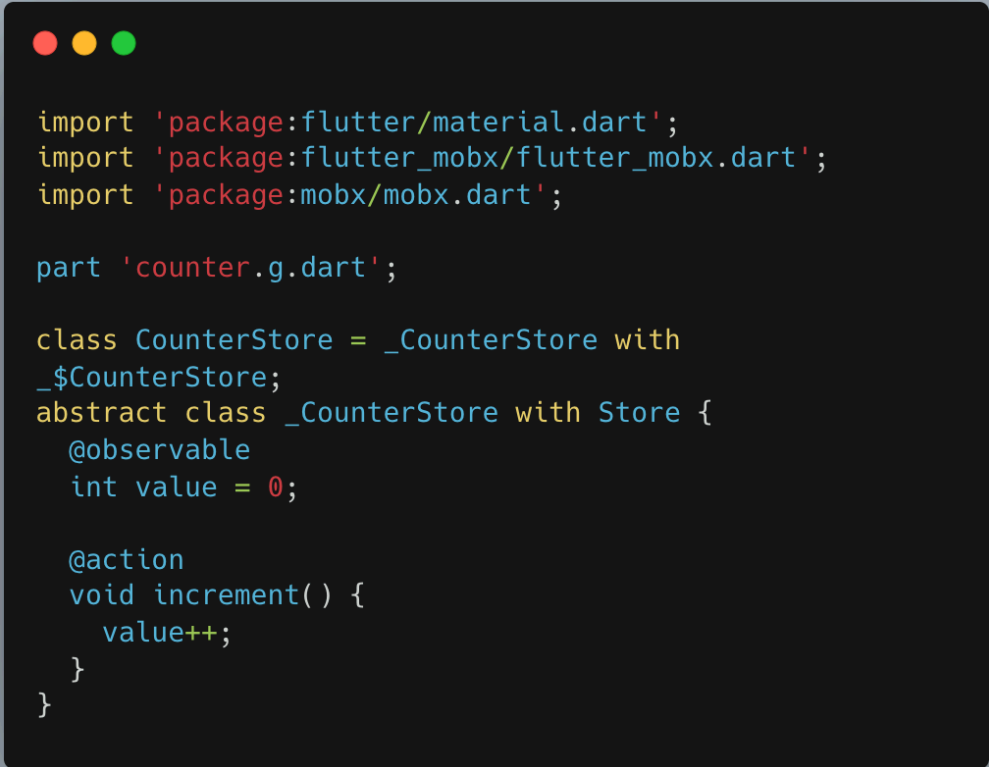
  @override
  Widget build(BuildContext context) {
    final TaskController controller =
    Get.put(TaskController());
    body: Center(
      child: Obx(() => Text('${controller.taskCount}')),
    ),
    floatingActionButton: FloatingActionButton(
      onPressed: controller.increment,
      child: const Icon(Icons.add),
    ),
  );
}

```

- **Ventajas:** Desarrollo extremadamente ágil con una mínima cantidad de código; sintaxis intuitiva; desacoplamiento total del contexto.
- **Desventajas:** Rompe con las convenciones arquitectónicas estándar de Flutter; centraliza demasiadas responsabilidades en una sola librería; comunidad dividida respecto a su mantenibilidad a largo plazo.
- **Casos de uso:** Startups, desarrollos ágiles (MVPs) y aplicaciones donde la velocidad de entrega en el mercado sea la máxima prioridad.

MobX

Basado en los principios del desarrollo web reactivo, MobX automatiza la sincronización entre los datos y la interfaz de usuario mediante el uso de observables, acciones y reacciones. Emplea la generación de código para interceptar las mutaciones y actualizar quirúrgicamente solo los widgets afectados.



```
import 'package:flutter/material.dart';
import 'package:flutter_mobx/flutter_mobx.dart';
import 'package:mobx/mobx.dart';

part 'counter.g.dart';

class CounterStore = _CounterStore with
  _$CounterStore;
abstract class _CounterStore with Store {
  @observable
  int value = 0;

  @action
  void increment() {
    value++;
  }
}
```

- **Ventajas:** Reactividad precisa a nivel microscópico; la interfaz se actualiza de forma transparente sin configuraciones manuales complejas.
- **Desventajas:** Dependencia absoluta del proceso de generación de código (build_runner); mensajes de error crípticos durante fallos de compilación.
- **Casos de uso:** Aplicaciones densas en datos interactivos en tiempo real, tableros analíticos y sistemas con un alto volumen de variables interconectadas.

6. Análisis Comparativo

Cuadro Comparativo de Métodos de Gestión de Estado

Método	Tipo de Enfoque	Dependencia de Contexto	Boilerplate	Seguridad de Tipos
setState()	Imperativo / Local	Alta (Interna)	Muy Bajo	Excelente
InheritedWidget	Declarativo / Propagación	Alta	Muy Alto	Excelente
Provider	Declarativo / Envoltorio	Alta	Bajo	Media (Riesgo en Ejecución)
Riverpod	Reactivo / Declarativo	Nula	Bajo	Excelente (Compilación)
BLoC / Cubit	Reactivo / Basado en Eventos	Alta	Alto	Excelente

GetX	Reactivo / Desacoplado	Nula	Mínimo	Baja
MobX	Reactivo / Generado	Nula	Medio	Excelente

Recomendación Técnica sobre qué Método Elegir

La selección de una arquitectura para la gestión de estados no debe basarse en tendencias, sino en los requisitos técnicos específicos, el tamaño del equipo y la longevidad proyectada del software.

1. **Prototipos Rápidos y Aplicaciones Pequeñas:** Para proyectos individuales o MVMs que deban validarse en el mercado en pocas semanas, **GetX** ofrece una velocidad operativa insuperable al unificar rutas, dependencias y estados con una sintaxis limpia. Si se prefiere mantener las directrices oficiales del ecosistema Flutter, la combinación de **setState** para la UI local y **Provider** para la persistencia global ligera resulta idónea.
2. **Aplicaciones Medianas a Grandes con Enfoque en Calidad:** Para el software comercial estándar, **Riverpod** se posiciona como la opción idónea en la actualidad. Su capacidad para detectar inconsistencias y errores de inyección durante la fase de compilación mitiga los fallos críticos en producción, al tiempo que su independencia del contexto simplifica drásticamente el desarrollo de pruebas automatizadas.
3. **Proyectos Empresariales Robustos:** En entornos corporativos donde múltiples equipos intervienen en la misma base de código, **BLoC** es el estándar indiscutible. La estructura rígida basada en eventos y estados secuenciales impone una disciplina de diseño uniforme, asegurando que cualquier desarrollador pueda interpretar, auditar y extender la lógica de negocio escrita por terceros de manera predictiva.

7. Conclusiones

La investigación detallada de las alternativas de gestión de estados en Flutter demuestra que el framework posee un ecosistema de desarrollo maduro y sumamente flexible, capaz de adaptarse a diversas filosofías de programación. Se concluye que no existe una solución absoluta o universal; cada herramienta fue diseñada para resolver deficiencias específicas de sus predecesoras o para priorizar un aspecto de desarrollo concreto, ya sea la velocidad de codificación, la robustez arquitectónica o el aislamiento de componentes.

Desde una perspectiva crítica e ingenieril, la mejor práctica en la gestión de estados consiste en aplicar el principio de responsabilidad única y buscar el mayor desacoplamiento posible entre las capas del software. La lógica de negocio jamás debería estar incrustada directamente dentro de los widgets de la interfaz visual. Asimismo, es perfectamente válido y recomendable combinar enfoques dentro de un mismo sistema: emplear mecanismos sencillos como `setState` para controlar transiciones estéticas o estados locales efímeros de las pantallas, y delegar en arquitecturas formales como BLoC o Riverpod la gestión de las reglas del negocio globales. La elección final debe guiarse rigurosamente por el balance óptimo entre la mantenibilidad del código y la velocidad de ejecución.

8. Referencias

- **Documentación Oficial de Flutter (Gestión de Estados):**
Flutter Dev. (s.f.). *State management*. Sitio web oficial de Flutter.
Link: <https://flutter.dev/development>
- **Librería Oficial de Provider:**
Pub.dev. (s.f.). *provider: A wrapper around InheritedWidget to make them easier to use and more reusable*. Repositorio oficial de paquetes para Flutter.
Link: <https://pub.dev/packages/provider>
- **Librería Oficial de Riverpod:**
Riverpod Team. (s.f.). *Riverpod: A reactive state-management framework for Flutter and Dart*. Sitio web oficial de la documentación de Riverpod.
Link: <https://riverpod.dev>

- **Librería Oficial de BLoC / Cubit:**

Bloc library authors. (s.f.). *Bloc State Management Library*. Sitio web oficial del patrón y ecosistema BLoC.

Link: <https://bloclibrary.dev>

- **Librería Oficial de MobX para Flutter:**

Pub.dev. (s.f.). *flutter_mobx: MobX integration for Flutter apps*. Repositorio oficial de paquetes para Flutter.

Link: https://pub.dev/packages/flutter_mobx